PROJECT THREE DISTRIBUTED FILE SYSTEM DESIGN



Mini-DFS: one NameServer and four DataServers

Chunk files, store three replicas and read by offset

Deadline: Friday, 6 November 2026

PROVIDED FILES AND YOUR TASKS

CANVAS: project3-student-kit.zip

Provided	You implement
Threads, Socket helpers and disk IO	Binary file chunking and balanced placement
Configuration and command-line client	Upload and file metadata
Data generation and checking scripts	Offset lookup and range reads
README and API reference	Verified replica reads and checksum repair

If you have already completed this project by following the original PDF,
you do not need to use this new guide or the ZIP package.

Expected: Read README.md first. Six core methods remain unfinished.



ORIGINAL PROJECT REQUIREMENTS

BASELINE FROM THE PROJECT BRIEF

Requirement	Expected behavior
Service structure	One NameServer and four DataServer threads
Upload	Store file bytes and return a file ID
Striping	2 MB chunks, distributed across four servers
Replication	Three copies on different servers
Lookup and read	File ID and offset locate the required content



ONE PROCESS AND FIVE SERVICE THREADS

RUNTIME STRUCTURE

Part	Where it runs	Responsibility
NameServer	One thread in the service JVM	Metadata and client commands
DataServer ds1 to ds4	Four threads in the same JVM	One local storage folder each
CLI client	Another terminal on the same VM	Send commands by Socket



CHUNKS AND REPLICAS

A FILE IS SPLIT BEFORE IT IS COPIED

Example	Meaning
4 MiB + 17-byte file	Three logical chunks: 2 MiB, 2 MiB, 17 bytes
Three replicas per chunk	Nine stored chunk copies in total
One short final chunk	Store 17 actual bytes for the tail
Four storage folders	Spread the nine copies across these folders



WHAT THE NAMESERVER REMEMBERS

FILE METADATA

Metadata	Example or purpose
Mini-DFS path and file ID	/read.bin maps to a generated ID
Original length	Used to check the reconstructed file
Ordered chunk list	Chunk 0, chunk 1, and so on
Chunk length and MD5	Check the bytes of each replica
Three server IDs per chunk	Locate copies on ds1 to ds4



BALANCED REPLICA PLACEMENT

THREE DIFFERENT LEAST-LOADED SERVERS

Measure the stored bytes on all online DataServers.

Choose three different servers with the lowest loads.

Finish the chunk writes, then refresh loads for the next chunk.

Use a stable server-ID order when loads are equal.

Expected: Balance stored bytes across four servers; one chunk still has exactly three copies.



UPLOAD FLOW

FROM LOCAL FILE TO MINI-DFS METADATA

Step	Action
1	Validate the source and target Mini-DFS name
2	Split the binary file into ordered chunks
3	Choose three servers and write every replica
4	Record chunk lengths, MD5 and replica locations
5	Publish file metadata and return the file ID



FILE READ AND MD5 CHECKS

FOLLOW THE ORDERED CHUNK LIST

Resolve the name or ID to file metadata.

Read a replica and compare its length and MD5 with metadata.

Use verified bytes and repair a bad online copy in the reference path.

Write each logical chunk once, in order, to the local output.



FILE ID AND BYTE OFFSET

ZERO-BASED LOCATION LOOKUP

```
chunk size        = 2,097,152 bytes
chunk index       = byte offset / chunk size   (integer division)
offset in chunk   = byte offset % chunk size

Example: byte offset 2,097,157
chunk index = 1
offset in chunk = 5
```

Expected: A location identifies the logical chunk and its three replica servers.



READS THAT CROSS A CHUNK BOUNDARY

RETURN THE REQUESTED CONTENT

```
Requested start: 2,097,144    Requested length: 32 bytes

Read 8 bytes from the end of chunk 0.
Read 24 bytes from the beginning of chunk 1.
Return the combined 32 bytes in that order.

Near EOF: return only the bytes that still exist.
```




BASELINE AND ORIGINAL BONUS TASKS

KEEP THE TWO LEVELS CLEAR

Baseline	Optional bonus from slide 6
Upload, download and file IDs	Hierarchical directory management
2 MB chunks and three replicas	Upload into a selected nested directory
Existence and offset content queries	Access by directory plus file name
Replica MD5 consistency	Lose one server and restore three live replicas



GROUP WORK AND COURSE ASSESSMENT

THE SAME COURSE ARRANGEMENT

Form groups of 4 to 6 members.

Every group completes Projects 1 to 4, with equal project weights.

Share environment, implementation, tests and report work.

Course grade: participation 20% and project work 80%.

Project work: team project 50% and functional check 30%.



REFERENCE ENVIRONMENT

REUSE YOUR EXISTING LINUX SETUP

Item	Example route
Operating system	Ubuntu 22.04 on an existing VM
Language	Java 8 in this optional starter
Service	One JVM with five service threads
Client	Another terminal on the same VM
Tools	JDK, Python 3 and unzip

Expected: Original slide 7 lists three Linux PCs/VMs; slide 3 defines the one-process service.



INSTALL AND CHECK TOOLS

ON THE LINUX VM

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository -y universe
sudo apt update
sudo apt install -y openjdk-8-jdk python3 unzip
java -version
javac -version
python3 --version
```

Expected: Use Java and javac 1.8 for this route. No external Java dependency is required.



EXTRACT THE LAB PACKAGE

A FRESH PROJECT DIRECTORY

```
mkdir -p ~/project3
unzip ~/project3-student-kit.zip -d ~/project3
cd ~/project3
less README.md
find src config scripts tests -type f | sort
```

Expected: src/, config/, scripts/, tests/ and README.md appear directly under ~/project3.



FOUR LOCAL STORAGE DIRECTORIES

config/local.properties

```
bind=127.0.0.1
port=9100
storage.root=storage

# Runtime folders:
storage/ds1/
storage/ds2/
storage/ds3/
storage/ds4/
```

Expected: Each DataServer writes actual chunk files into its own directory.



BUILD THE STUDENT SKELETON

FROM ~/project3

```
bash scripts/build.sh
jar tf build/minidfs.jar | head
java -cp build/classes mini.CoreSelfTest
```

Expected: Compilation succeeds. The untouched core test fails with TODO 1.



TODO 1 BINARY FILE CHUNKING

src/mini/Chunking.java

```
split(localSource) -> ordered List<byte[]>

Read up to CHUNK_BYTES into each chunk.
Continue reading until the chunk is full or the file ends.
Keep the actual byte count for the final short chunk.
Concatenate the chunks to reproduce the original file.
```




TODO 2 REPLICA PLACEMENT

src/mini/Placement.java

```
choose(currentByteLoads, 3) -> three DISTINCT server IDs

Consider online servers only.
Prefer lower stored-byte counts.
Break equal-load ties by server ID.
Reject a request if fewer than three servers are available.
```




TODO 3 UPLOAD AND METADATA

src/mini/NameServer.java: upload

Validate the source and reject an existing destination name.

Create a file ID and split the source into chunks.

For every chunk, choose servers, write copies and record its MD5.

After all writes succeed, publish path and file-ID metadata.



TODO 4 LOCATION LOOKUP

src/mini/NameServer.java: locate

Find the file record by file ID.

Check that the offset identifies an existing byte.

Calculate chunk index and offset inside that chunk.

Return the chunk metadata, including its replica locations.



TODO 5 CONTENT AT AN OFFSET

src/mini/NameServer.java: readRange

Validate the file, starting offset and requested length.

Read only the needed part of the first chunk.

Continue through later chunks until the request or file ends.

Return exact binary bytes in their original order.



TODO 6 VERIFIED REPLICA READS

src/mini/NameServer.java: readVerifiedChunk

Compare replica length and MD5 with trusted chunk metadata.

Keep a verified good copy and identify invalid online copies.

Repair bad online copies from the verified bytes.

If no correct replica exists, report a read failure.



GENERATE DEMONSTRATION FILES

LARGER THAN THE ORIGINAL SIZE THRESHOLDS

```
python3 scripts/make_data.py
cat data/manifest.json

# write-4m.bin = 4 MiB + 17 bytes  (> 3 MB)
# read-8m.bin  = 8 MiB + 123 bytes (> 7 MB)
```

Expected: The examples produce 3 and 5 logical chunks with short final chunks.



START THE MINIDFS SERVICE

KEEP THIS TERMINAL OPEN

```
bash scripts/run.sh

# Expected startup lines:
START DataServer-ds1 ...
START DataServer-ds2 ...
START DataServer-ds3 ...
START DataServer-ds4 ...
START NameServer port=9100

# Another terminal:
bash scripts/cli.sh ping
```

Expected: PING returns PONG. Core commands still need the student implementations.



UPLOAD A FILE LARGER THAN 3 MB

CLIENT TERMINAL ON THE SAME VM

```
bash scripts/cli.sh put data/write-4m.bin /write.bin
bash scripts/cli.sh stat /write.bin
bash scripts/cli.sh status
```

Expected: UPLOADED, a nonempty fileId, 4,194,321 bytes and 3 logical chunks.



READ A FILE LARGER THAN 7 MB

REASSEMBLE THE COMPLETE FILE

```
bash scripts/cli.sh put data/read-8m.bin /read.bin
bash scripts/cli.sh get /read.bin recovered/read.bin
cmp data/read-8m.bin recovered/read.bin
md5sum data/read-8m.bin recovered/read.bin
```

Expected: The two files have 8,388,731 bytes and identical content.



FILE EXISTENCE AND METADATA

QUERY BY A GIVEN MINIDFS NAME

```
bash scripts/cli.sh list
bash scripts/cli.sh exists /read.bin
bash scripts/cli.sh exists /missing.bin
bash scripts/cli.sh stat /read.bin
```

Expected: The uploaded path exists, the missing path does not, and metadata lists ordered chunks.



OFFSET LOCATION AND CONTENT

COMPARE WITH THE ORIGINAL LOCAL BYTES

```
# Replace FILE_ID with the ID printed by PUT.
bash scripts/cli.sh locate FILE_ID 2097157
bash scripts/cli.sh read /read.bin 2097144 32

python3 - <<'PY'
from pathlib import Path
b = Path('data/read-8m.bin').read_bytes()
print(b[2097144:2097176].hex())
PY
```

Expected: LOCATE returns chunk 1, offset 5. READ hex matches the local slice.



REPLICA MD5 AND SERVER LOADS

EXAMINE THE ACTUAL STORED CHUNKS

```
bash scripts/cli.sh verify /read.bin
bash scripts/cli.sh status
find storage -name '*.chunk' -type f | sort

# Inspect an actual chunk file if needed:
# md5sum storage/ds1/CHUNK_ID.chunk
```

Expected: Each chunk has 3 matching copies. The example workload is spread across all 4 servers.



OPTIONAL SYSTEM CHECKER

REPLAY THE ORIGINAL FUNCTIONAL REQUIREMENTS

```
python3 tests/check_system.py --out evidence/check.json

# Optional reference read-repair check from original slide 11:
python3 tests/check_system.py --integrity   --out evidence/check-integrity.json
```

Expected: PASS after the core methods work. Inspect the saved requests, replies and timings.



ERRORS AND COMMON PROBLEMS

CHECK THE FIRST FAILING STEP

Symptom	Likely check
TODO reply	One or more core methods are unfinished
Wrong downloaded size	Tail length, order, or repeated replicas
Wrong offset content	Byte offset, in-chunk offset, boundary copying
One server stores too much	Refresh loads after each chunk
MD5 mismatch	Compare actual bytes with trusted metadata
File missing after restart	Metadata was in memory; use a fresh session



OPTIONAL BONUS DIRECTORY MANAGEMENT

ORIGINAL PROJECT SLIDE 6

```
# After implementing the directory extension:
bash scripts/cli.sh mkdir /course
bash scripts/cli.sh put data/write-4m.bin /course/write.bin
bash scripts/cli.sh exists /course/write.bin
bash scripts/cli.sh get /course/write.bin recovered/bonus-dir.bin
```

Expected: Directory plus file name resolves to the correct stored file.



OPTIONAL BONUS SERVER-LOSS RECOVERY

FOUR SERVERS BECOME THREE

Make one DataServer unavailable and exclude it from placement.

Find each chunk that lost a replica.

Copy verified surviving bytes onto a missing live server.

Update metadata and check three distinct live copies per chunk.

Expected: With 3 servers alive, every logical chunk must have a copy on each of them.



REPORT AND PERFORMANCE ANALYSIS

USE YOUR ACTUAL RESULTS

Explain the environment, threads, metadata and core implementation.

Show file equality, chunk locations, replica MD5 and error handling.

Analyse chunk distribution and storage cost.

Optionally measure upload, download or range-read time.

State file sizes, test conditions and the limits of your conclusions.

```
python3 scripts/measure.py   # optional end-to-end timing helper
```




PROJECT 3 GRADING

COURSE GRADING RUBRIC

Category	Criterion	Weight
Basic functionality	Correct source-code functions	20%
Basic functionality	Correct outputs	30%
Basic functionality	Exception handling	10%
Report analysis	Report quality	20%
Report analysis	Source-code performance analysis	10%
Enhanced functions	Directory or server-loss recovery	10% bonus

Expected: Basic functionality 60%, report analysis 30%, enhanced functions 10% bonus.



EXPECTED DEMONSTRATION RESULTS

CHECK AGAINST ORIGINAL SLIDE 5

Original request	Evidence to show
Write a file larger than 3 MB	Successful upload and returned file ID
Read a file larger than 7 MB	Downloaded bytes equal the source
Appropriate chunk distribution	Counts and bytes across four servers
Same chunk, same replica MD5	Actual per-replica checksum results
File exists or does not exist	Both root-path query outcomes
File plus random offset	Correct location and returned bytes



BEFORE SUBMISSION

SOURCE, REPORT AND ACTUAL EVIDENCE

Rebuild the submitted source and demonstrate the required functions.

Include configuration, commands, results and member contributions.

Identify optional bonus work separately from baseline results.

Exclude generated large data and private credentials from the package.

Follow Canvas for the required submission format and location.



CANVAS, HELP AND DEADLINE

FINAL REMINDERS

Check Canvas for the slides, optional lab ZIP and submission instructions.

Deadline: Friday, 6 November 2026. Check the exact cutoff on Canvas.

Ask AI or the TA for explanations, and verify the proposed changes.

When asking for help, include the command, full error and relevant code.

